

```

// Ruifeng Song 06/02/2025
#include "Particle.h"

// Constants
const size_t read_buf_size = 300; // Increased buffer size to accommodate
large chunks of data

// Buffer and index for incoming data from Serial1
char readBuf[read_buf_size];
size_t read_buf_index = 0;

SYSTEM_MODE(AUTOMATIC); // Automatic management of cloud connections

void setup() {
    Serial1.begin(9600); // Start serial communication on Serial1 at 9600
    baud
    Particle.publish("Setup", "Setup Done", PRIVATE); // Publish a test
    event with test data
    Serial1.flush();
    Serial1.print("$"); // tell Arduino ready to receive data
}

void loop() {
    receiveBuffer(); // Handle incoming data from Serial1
}

void receiveBuffer() {
    while (Serial1.available() > 0) { // Check if data is available on
    Serial1
        char c = Serial1.read(); // Read a character from Serial1

        if (read_buf_index < read_buf_size - 1) {
            if (c == ';') { // Check for semicolon indicating
the end of the data
                readBuf[read_buf_index] = '\0'; // Null-terminate the string
                Particle.publish("Environmentdata", readBuf, PRIVATE); // Publish
the complete message
                delay(5000);
                read_buf_index = 0; // Reset buffer index for the next
sample
                Serial1.flush();
                Serial1.print("#"); // tell Arduino ready to turn off
            } else {
                readBuf[read_buf_index++] = c; // Add character to readBuf
            }
        } else {
            Particle.publish("Error", "readBuf overflow, emptying buffer",
PRIVATE); // Publish error, buffer overflow
            read_buf_index = 0; // Reset index if overflow occurs
        }
    }
}

// Ruifeng Song

```